

porting-manual

STUDION 포팅 매뉴얼

1. 문서 개요

본 문서는 STUDION 프로젝트를 신규 서버 또는 재현 환경에 포팅하기 위한 설치, 설정, 빌드, 배포 절차를 정리한다.

- 프로젝트명: STUDION
- 서비스 개요: 음악 프로젝트 협업, 오디오 업로드/버전 관리, 실시간 편집 상태 동기화, AI 기반 오디오 분석/제안 서비스
- 주요 구성: Vue 3 프론트엔드, Spring Boot 백엔드, FastAPI AI 서버, MySQL, MongoDB, Redis, Nginx, Jenkins
- 기준 브랜치: `release`
- 운영 도메인 예시: `studion.ai.kr`

2. 시스템 구성

2.1 서비스 구성

구분	경로	기술 스택	기본 포트	역할
Frontend	FE	Vue 3, TypeScript, Vite, Pinia, Tailwind CSS	개발 5173 , 운영 80/443	사용자 화면, 오디오 편집 UI, WebSocket 클라이언트
Backend	BE	Java 21, Spring Boot 3.5.13, Gradle 8.14.4	8080	REST API, OAuth2/JWT 인증, WebSocket, S3 연동, DB 영속화
AI	AI	Python 3.11, FastAPI, LangGraph, Dramatiq, Redis	8000	AI 워크플로우 실행, 분석 결과/프리뷰 생성
MySQL	외부 DB 또는 개발 Docker	MySQL 8.0	개발 3307:3306	핵심 관계형 데이터 저장
MongoDB	외부 DB 또는 개발 Docker	MongoDB 8.0	개발 27017	이벤트/스냅샷/AI 아티팩트 저장
Redis	Docker	Redis 7	개발 6379 , 운영 127.0.0.1:6379	세션, 임시 상태, 락, 큐

구분	경로	기술 스택	기본 포트	역할
Nginx	FE/nginx.conf	Nginx 1.27 Alpine	80 , 443	정적 파일 서빙, HTTPS, API/WebSocket reverse proxy

2.2 운영 배포 구조

운영 서버는 **인프라 서버**와 **AI 서버** 두 대로 분리되어 있다.

구분	실행 compose 파일	컨테이너	비고
인프라 서버	compose.prod.yaml	nginx , backend , redis , certbot	외부 요청 수신, SSL 종단
AI 서버	compose.ai.yaml	ai-api , ai-worker , ai- redis	별도 서버에서 독립 실행

서버 간 통신 흐름은 다음과 같다.



- Nginx는 AI 서버를 직접 프록시하지 않는다. AI 서버는 백엔드가 FASTAPI_URL 로 서버 내부에서 직접 호출한다.
- 운영 MySQL/MongoDB는 compose에 포함되어 있지 않으므로 외부 DB 또는 별도 서버를 준비한다.
- AI 서비스는 Jenkins의 ai-server 라벨 agent에서 별도로 빌드/배포된다.

2.3 서버 간 네트워크 요구사항

방향	프로토콜/포트	비고
클라이언트 → 인프라 서버	TCP 80, 443	Nginx 수신
인프라 서버 → AI 서버	TCP 8000	백엔드 → ai-api 호출
인프라 서버 → 외부 DB	TCP 3306, 27017	MySQL, MongoDB
AI 서버 → 외부 DB	TCP 3306, 27017	MySQL, MongoDB

AI 서버의 8000 포트는 인프라 서버 IP에서만 접근 가능하도록 방화벽(보안 그룹)을 설정한다. 불특정 외부에서 ai-api 에 직접 접근하는 것을 차단해야 한다.

2.4 기술 스택 상세 버전

런타임 및 서버

구분	제품	버전	비고
JVM (빌드)	BellSoft Liberica JDK	21	<code>.idea/misc.xml</code> 기준 로컬 개발 JDK
JVM (운영)	Eclipse Temurin JRE	21	<code>BE/Dockerfile</code> runtime stage
WAS	Spring Boot 내장 Tomcat	Spring Boot 3.5.13 포함	별도 Tomcat 설치 불필요
웹서버	Nginx	1.27-alpine	정적 파일 서빙 + reverse proxy
AI 런타임	Python	3.11-slim	<code>AI/Dockerfile</code> 기준

빌드 도구

구분	도구	버전
백엔드 빌드	Gradle	8.14.4 (<code>gradle/wrapper/gradle-wrapper.properties</code>)
프론트엔드 패키지	npm	Node.js 22 기본 포함
AI 패키지	uv	최신 (<code>pip install uv</code> 로 설치)

주요 라이브러리 버전

구분	라이브러리	버전
BE	Spring Boot	3.5.13
BE	Spring Dependency Management	1.1.7
BE	AWS SDK S3	2.31.13
BE	jjwt	0.12.6

구분	라이브러리	버전
BE	springdoc-openapi	2.8.6
BE	MapStruct	1.6.3
BE	QueryDSL	5.1.0
BE	Flyway	Spring Boot 관리 버전
FE	Vue	3.5.32
FE	Vite	8.0.10
FE	TypeScript	
FE	Pinia	3.0.4
FE	Tailwind CSS	4.2.4
FE	Tone.js	15.1.22
FE	STOMP.js	^7.3.0
AI	FastAPI	≥0.115
AI	LangGraph	≥0.4
AI	Dramatiq[Redis]	≥1.17
AI	SQLAlchemy	≥2.0

IDE 권장 설정

역할	권장 IDE	설정
백엔드 개발	IntelliJ IDEA 2024.x 이상	Project SDK: Liberica JDK 21 (.idea/misc.xml 참고)
프론트엔드 개발	VS Code 또는 WebStorm	Volar (Vue) 확장 설치 권장
AI 개발	VS Code	Python 3.11 인터프리터 설정, Ruff 확장 설치 권장

3. 사전 준비

3.1 서버 요구사항

인프라 서버와 AI 서버 각각에 필요한 프로그램은 다음과 같다.

인프라 서버

항목	권장 버전	비고
OS	Ubuntu 22.04 LTS 이상	
Docker	24.x 이상	

항목	권장 버전	비고
Docker Compose	v2.x	
Git	2.x	
Jenkins	LTS	CI/CD 파이프라인 실행
Java	21	백엔드 로컬 빌드 시 필요
Node.js	22	프론트엔드 로컬 빌드 시 필요

AI 서버

항목	권장 버전	비고
OS	Ubuntu 22.04 LTS 이상	
Docker	24.x 이상	
Docker Compose	v2.x	
Git	2.x	
Python	3.11	AI 로컬 실행 시 필요
uv	최신	AI 의존성 설치/실행 시 필요
Jenkins agent	LTS	Jenkins에서 원격 배포 시 필요 (8.1 참고)

3.2 외부 서비스 준비

다음 외부 리소스를 미리 생성한다.

- MySQL 8.0 데이터베이스
- MongoDB 데이터베이스
- AWS S3 버킷
- CDN 도메인 또는 S3 접근 도메인
- Google OAuth 2.0 Client ID/Secret
- Let's Encrypt 인증서 발급 대상 도메인
- AI에서 사용할 외부 CLAP 추론 서버 또는 비활성화 설정
- 선택 사항: Planning LLM, Plan Critic LLM API 키

4. 저장소 클론

```
git clone <저장소_URL> S14P31A205
cd S14P31A205
git checkout release
```

5. 환경 변수 설정

비밀 값은 Git에 커밋하지 않는다. 운영 배포에서는 Jenkins Credentials 또는 서버의 안전한 경로에서 `.env.prod`, `.env.ai` 를 주입한다.

5.1 백엔드/프론트 운영 환경 변수: `.env.prod`

루트에 `.env.prod` 파일을 생성한다.

```
# 서버 공통
SPRING_PROFILES_ACTIVE=prod
FASTAPI_URL=http://<AI_SERVER_HOST>:8000/api/v1
CDN_DOMAIN=<CDN_DOMAIN>
VITE_API_BASE_URL=https://studion.ai.kr
VITE_WS_BASE_URL=wss://studion.ai.kr
FRONTEND_URL=https://studion.ai.kr

# MySQL
MYSQL_DATABASE=<MYSQL_DATABASE>
MYSQL_USER=<MYSQL_USER>
MYSQL_PASSWORD=<MYSQL_PASSWORD>
MYSQL_ROOT_PASSWORD=<MYSQL_ROOT_PASSWORD>
DB_URL=jdbc:mysql://<MYSQL_HOST>:3306/<MYSQL_DATABASE>?
serverTimezone=UTC&characterEncoding=UTF-8
DB_USERNAME=<DB_USERNAME>
DB_PASSWORD=<DB_PASSWORD>

# MongoDB
MONGO_USERNAME=<MONGO_USERNAME>
MONGO_PASSWORD=<MONGO_PASSWORD>
MONGO_DB=<MONGO_DB>
MONGODB_DB=<MONGODB_DB>
MONGODB_PORT=27017
MONGODB_URI=mongodb://<MONGO_USERNAME>:
<MONGO_PASSWORD>@<MONGO_HOST>:27017/<MONGODB_DB>?authSource=admin

# Redis
REDIS_HOST=redis
REDIS_PORT=6379
REDIS_PASSWORD=<REDIS_PASSWORD>

# JWT
JWT_SECRET=<충분히 긴 랜덤 문자열>
JWT_EXPIRATION=86400000
JWT_REFRESH_EXPIRATION=1209600000
JWT_TMP_EXPIRATION=300000

# Google OAuth 2.0
GOOGLE_CLIENT_ID=<GOOGLE_CLIENT_ID>
GOOGLE_CLIENT_SECRET=<GOOGLE_CLIENT_SECRET>
```

```
# AWS S3
AWS_ACCESS_KEY=<AWS_ACCESS_KEY>
AWS_SECRET_KEY=<AWS_SECRET_KEY>
AWS_S3_BUCKET=<AWS_S3_BUCKET>
AWS_UPLOAD_URL_EXPIRATION_MINUTES=10

# 기타 API Key
GMS_API_KEY=<GMS_API_KEY>

# Certbot
CERTBOT_EMAIL=<관리자_이메일>
CERTBOT_DOMAIN=studio.ai.kr

# DB Pool
DB_MAX_POOL_SIZE=100
DB_MIN_IDLE=5
DB_CONNECTION_TIMEOUT=30000
DB_IDLE_TIMEOUT=60000
```

5.2 개발 환경 변수: .env

개발 환경은 `compose.dev.yaml` 로 MySQL, MongoDB, Redis를 실행한다.

```
SPRING_PROFILES_ACTIVE=dev
SERVER_PORT=8080
FASTAPI_URL=http://localhost:8000/api/v1
FRONTEND_URL=http://localhost:5173
CDN_DOMAIN=<CDN_DOMAIN>

DB_URL=jdbc:mysql://localhost:3307/<MYSQL_DATABASE>?
serverTimezone=UTC&characterEncoding=UTF-8
DB_USERNAME=<DB_USERNAME>
DB_PASSWORD=<DB_PASSWORD>

MONGODB_DB=<MONGODB_DB>
MONGODB_PORT=27017
REDIS_PORT=6379

AWS_ACCESS_KEY=<AWS_ACCESS_KEY>
AWS_SECRET_KEY=<AWS_SECRET_KEY>
AWS_S3_BUCKET=<AWS_S3_BUCKET>
AWS_UPLOAD_URL_EXPIRATION_MINUTES=10
GMS_API_KEY=<GMS_API_KEY>

GOOGLE_CLIENT_ID=<GOOGLE_CLIENT_ID>
GOOGLE_CLIENT_SECRET=<GOOGLE_CLIENT_SECRET>
JWT_SECRET=<충분히 긴 랜덤 문자열>
```

5.3 AI 환경 변수: `.env.ai`

AI 서버는 `STUDION_AI_` prefix를 사용한다. 루트에 `.env.ai` 를 둔다.

```
STUDION_AI_ENVIRONMENT=prod
STUDION_AI_LOG_LEVEL=INFO
STUDION_AI_CORS_ALLOW_ORIGINS=https://studio.ai.kr
STUDION_AI_DRAMATIQ_BROKER_URL=redis://ai-redis:6379/0

STUDION_AI_MYSQL_HOST=<MYSQL_HOST>
STUDION_AI_MYSQL_PORT=3306
STUDION_AI_MYSQL_DATABASE=<MYSQL_DATABASE>
STUDION_AI_MYSQL_USER=<MYSQL_USER>
STUDION_AI_MYSQL_PASSWORD=<MYSQL_PASSWORD>

STUDION_AI_MONGO_URL=mongodb://<MONGO_USERNAME>:
<MONGO_PASSWORD>@<MONGO_HOST>:27017/<MONGODB_DB>?authSource=admin
STUDION_AI_MONGO_DATABASE=<MONGODB_DB>
STUDION_AI_MONGO_SNAPSHOT_COLLECTION=timeline_snapshots
STUDION_AI_MONGO_ARTIFACT_COLLECTION=workflow_artifacts

STUDION_AI_AUDIO_CACHE_DIR=/tmp/studio-ai-audio-cache
STUDION_AI_CLAP_ENABLED=true
STUDION_AI_CLAP_INFERENCE_URL=<CLAP_INFERENCE_URL>

STUDION_AI_PLANNING_LLM_ENABLED=false
STUDION_AI_PLANNING_LLM_BASE_URL=<LLM_BASE_URL>
STUDION_AI_PLANNING_LLM_API_KEY=<LLM_API_KEY>
STUDION_AI_PLANNING_LLM_MODEL=gpt-5.2

STUDION_AI_PLAN_CRITIC_ENABLED=false
STUDION_AI_PLAN_CRITIC_BASE_URL=<ANTHROPIC_BASE_URL>
STUDION_AI_PLAN_CRITIC_API_KEY=<ANTHROPIC_API_KEY>
STUDION_AI_PLAN_CRITIC_MODEL=claude-sonnet-4-5-20250929
```

6. 로컬 개발 실행

6.1 인프라 컨테이너 실행

```
docker compose --env-file .env -f compose.dev.yaml up -d
```

실행되는 컨테이너:

- `mysql` : `localhost:3307`
- `mongodb` : `localhost:27017`
- `redis` : `localhost:6379`

6.2 백엔드 실행

```
cd BE
./gradlew bootRun
```

Windows PowerShell에서는 다음 명령을 사용한다.

```
cd BE
.\gradlew.bat bootRun
```

백엔드 주요 경로:

- REST API: `http://localhost:8080/api/v1`
- Swagger UI: `http://localhost:8080/swagger-ui/index.html`
- WebSocket: `ws://localhost:8080/ws/projects/{projectId}`
- Chat WebSocket: `ws://localhost:8080/ws/chat`

6.3 프론트엔드 실행

```
cd FE
npm ci
npm run dev
```

접속 주소:

- `http://localhost:5173`

프론트엔드는 기본적으로 다음 값을 사용한다.

- `VITE_API_BASE_URL` 미설정 시 `http://localhost:8080`
- `VITE_WS_BASE_URL` 미설정 시 `ws://localhost:8080`

6.4 AI 서버 실행

```
cd AI
uv sync --group audio-dsp
uv run uvicorn app.main:app --host 0.0.0.0 --port 8000
```

AI worker는 별도 터미널에서 실행한다.

```
cd AI
uv run python -m dramatiq app.services.workflow_queue
```

AI 주요 경로:

- Health Check: `http://localhost:8000/api/v1/health`
- Workflow graph: `http://localhost:8000/api/v1/graph/workflow`
- Internal workflow start: `POST /api/v1/internal/workflow/jobs/start`

7. 운영 배포

7.1 SSL 인증서 발급

운영 도메인의 DNS A 레코드가 배포 서버를 바라보도록 설정한 뒤 최초 1회 인증서를 발급한다.

```
docker compose --env-file .env.prod -f compose.prod.yaml run --rm certbot
```

인증서 경로:

- `/etc/letsencrypt/live/studion.ai.kr/fullchain.pem`
- `/etc/letsencrypt/live/studion.ai.kr/privkey.pem`

7.2 애플리케이션 빌드 및 배포

```
docker compose --env-file .env.prod -f compose.prod.yaml build
docker compose --env-file .env.prod -f compose.prod.yaml up -d --remove-orphans
docker compose --env-file .env.prod -f compose.prod.yaml ps
```

7.3 AI 서버 빌드 및 배포

Jenkins CI/CD를 사용하는 경우 `release` 브랜치 푸시 시 AI 서버 배포가 자동으로 실행된다 (섹션 8 참고). 아래 명령은 Jenkins 없이 **AI 서버에서 직접 수동 배포**할 때 사용한다.

AI 서버에 SSH 접속한 뒤 저장소 디렉터리에서 실행한다.

```
cd /home/ec2-user/deploy/S14P31A205
git fetch origin release && git checkout release && git pull --ff-only origin release
docker compose --env-file .env.ai -f compose.ai.yaml build
docker compose --env-file .env.ai -f compose.ai.yaml up -d --scale ai-worker=3
docker compose --env-file .env.ai -f compose.ai.yaml ps
```

`.env.ai` 는 루트 디렉터리(`S14P31A205/.env.ai`)에 위치해야 한다.

7.4 배포 후 확인

```
curl -I https://studion.ai.kr
curl https://studion.ai.kr/api/v1/auth/positions
```

```
curl http://<AI_SERVER_HOST>:8000/api/v1/health
docker compose --env-file .env.prod -f compose.prod.yaml logs -f backend
```

8. Jenkins CI/CD

8.1 Jenkins agent 구성

파이프라인은 두 종류의 agent를 사용한다.

agent	실행 위치	역할
any (기본)	인프라 서버 (Jenkins 설치 서버)	백엔드/프론트 빌드, 인프라 서버 배포
ai-server label	AI 서버	AI 이미지 빌드 및 AI 서버 배포

ai-server label agent는 AI 서버에 Jenkins agent를 설치하고 인프라 서버의 Jenkins에 등록해야 한다.

AI 서버에 Jenkins agent 설치 절차

- 인프라 서버의 Jenkins 관리 페이지 → Manage Nodes → New Node 생성
 - Node name: 임의 지정
 - Label: ai-server
 - Remote root directory: /home/ec2-user/jenkins-agent
 - Launch method: Launch agent via SSH 또는 Launch agent by connecting it to the controller
- 생성된 노드의 연결 명령을 AI 서버에서 실행하여 agent를 등록한다.
- AI 서버에 저장소를 미리 클론해둔다.

```
# AI 서버에서 실행
git clone <저장소_URL> /home/ec2-user/deploy/S14P31A205
cd /home/ec2-user/deploy/S14P31A205
# .env.ai 파일도 이 위치에 생성
```

8.2 파이프라인 동작

루트 Jenkinsfile 기준 파이프라인은 다음 단계로 동작한다.

- Checkout : Jenkins workspace 초기화 후 SCM checkout
- CI - Backend Build : BE 에서 ./gradlew clean build
- CI - Frontend Build : FE Docker builder stage 빌드
- CI - AI Build : ai-server agent에서 release 브랜치 fetch/pull 후 AI compose build
- Prepare Env : Jenkins Credentials의 studion-prod-env 파일을 .env.prod 로 복사
- CD - Build App : compose.prod.yaml build
- CD - Deploy App : compose.prod.yaml up -d --remove-orphans

8. CD - Deploy AI : release 브랜치에서만 ai-server agent로 AI worker 3개 scale 배포
9. CD - Status App : 운영 compose 상태 확인

Jenkins 설정값:

항목	값
Pipeline 파일	Jenkinsfile
운영 env credential id	studion-prod-env
AI agent label	ai-server
Compose project name	studion
AI 배포 경로	/home/ec2-user/deploy/S14P31A205

9. 데이터베이스 초기화 및 마이그레이션

9.1 MySQL

백엔드는 Flyway를 사용한다.

- 마이그레이션 경로: BE/src/main/resources/db/migration
- Hibernate DDL 정책: validate
- Flyway baseline: baseline-on-migrate=true , baseline-version=0
- 마이그레이션 파일 목록 및 전체 스키마/시드 SQL은 섹션 17 참고

운영 DB는 애플리케이션 시작 시 Flyway가 마이그레이션을 적용할 수 있어야 한다. DB 계정에는 스키마 변경 권한이 필요하다.

9.2 MongoDB

MongoDB는 프로젝트 이벤트, 트랙/클립 이벤트 문서, AI 스냅샷과 아티팩트 저장에 사용한다. 별도 migration 스크립트는 없으며 애플리케이션이 필요한 컬렉션을 사용한다.

9.3 Redis

Redis는 세션, 프로젝트 초대 코드, EQ/리미터 락과 draft 상태, AI 큐에 사용한다. 운영 compose.prod.yaml 의 Redis는 password를 사용한다.

10. 주요 API 및 프록시 경로

10.1 Nginx 프록시

운영 Nginx 설정은 FE/nginx.conf 에 있다.

외부 경로	내부 대상
/	Vue SPA 정적 파일

외부 경로	내부 대상
/api/	http://backend:8080
/oauth2/	http://backend:8080/oauth2/
/login/oauth2/	http://backend:8080/login/oauth2/
/ws/	http://backend:8080/ws/
/.well-known/acme-challenge/	Certbot webroot

10.2 백엔드 주요 API

기능	경로
인증/온보딩	/api/v1/auth/*
프로젝트	/api/v1/projects/*
프로젝트 초대	/api/v1/projects/{projectId}/invitations , /api/v1/invitations/{inviteCode}/accept
오디오	/api/v1/projects/{projectId}/audios/*
오디오 버전	/api/v1/projects/{projectId}/versions/*
댓글	/api/v1/projects/{projectId}/comments
EQ	/api/v1/eq/*
Master Limiter	/api/v1/limiter/*
AI workflow	/api/v1/ai/workflow/jobs/*

10.3 WebSocket

기능	경로
프로젝트 협업 이벤트	/ws/projects/{projectId}?accessToken=<JWT>
채팅	/ws/chat

11. 빌드 및 테스트 명령

11.1 Backend

```
cd BE
./gradlew clean build
./gradlew test
```

11.2 Frontend

```
cd FE
npm ci
npm run lint
npm run type-check
npm run build
```

11.3 AI

```
cd AI
uv sync --group audio-dsp --group dev
uv run pytest
uv run ruff check .
```

12. 운영 점검 체크리스트

- DNS A 레코드가 운영 서버 IP를 가리키는지 확인한다.
- CERTBOT_DOMAIN 과 Nginx server_name 이 같은지 확인한다.
- /etc/letsencrypt 가 Docker volume mount 대상 서버에 존재하는지 확인한다.
- .env.prod 의 VITE_API_BASE_URL 은 https://studion.ai.kr 형식으로 설정한다.
- .env.prod 의 VITE_WS_BASE_URL 은 wss://studion.ai.kr 형식으로 설정한다.
- 백엔드 FASTAPI_URL 이 실제 AI API 주소를 가리키는지 확인한다.
- MySQL 계정에 Flyway migration 권한이 있는지 확인한다.
- MongoDB URI의 인증 DB(authSource)가 실제 계정 생성 위치와 일치하는지 확인한다.
- Redis password가 .env.prod 와 compose Redis command에 동일하게 적용되는지 확인한다.
- S3 버킷 CORS와 IAM 권한이 업로드/다운로드 정책에 맞는지 확인한다.
- Google OAuth Redirect URI에 운영 콜백 URL을 등록한다.

13. 장애 대응

13.1 프론트 접속은 되지만 API가 실패하는 경우

확인 항목:

- docker compose --env-file .env.prod -f compose.prod.yaml ps
- docker compose --env-file .env.prod -f compose.prod.yaml logs -f backend
- FE/nginx.conf 의 /api/ proxy 대상이 backend:8080 인지 확인
- VITE_API_BASE_URL 이 빌드 시점에 올바르게 들어갔는지 확인

13.2 WebSocket 연결이 실패하는 경우

확인 항목:

- 프론트 VITE_WS_BASE_URL 이 wss://studion.ai.kr 인지 확인

- Nginx /ws/ location에 Upgrade , Connection 헤더가 있는지 확인
- 클라이언트가 /ws/projects/{projectId}?accessToken=<JWT> 형식으로 연결하는지 확인
- JWT 만료 또는 잘못된 프로젝트 권한 여부 확인

13.3 백엔드가 시작되지 않는 경우

확인 항목:

- MySQL DB_URL , DB_USERNAME , DB_PASSWORD
- MongoDB MONGODB_URI , MONGODB_DB
- Redis REDIS_HOST , REDIS_PORT , REDIS_PASSWORD
- Flyway migration 실패 로그
- S3, JWT, Google OAuth 필수 환경 변수 누락 여부

13.4 AI workflow가 진행되지 않는 경우

확인 항목:

- ai-api , ai-worker , ai-redis 컨테이너 상태
- STUDIO_AI_DRAMATIQ_BROKER_URL 이 redis://ai-redis:6379/0 인지 확인
- AI worker scale이 1개 이상인지 확인
- 백엔드의 AI 서버 URL이 http://<AI_SERVER_HOST>:8000/api/v1 형식인지 확인
- CLAP 또는 LLM 외부 API 사용 시 API URL/Key와 timeout 로그 확인

13.5 SSL 인증서 발급 실패

확인 항목:

- 도메인 DNS가 현재 서버를 가리키는지 확인
- 80번 포트가 외부에서 접근 가능한지 확인
- CERTBOT_EMAIL , CERTBOT_DOMAIN 값 확인
- /.well-known/acme-challenge/ 경로가 Nginx에서 /var/www/certbot 으로 연결되는지 확인

14. 주요 파일 위치

파일	설명
compose.dev.yaml	개발용 MySQL, MongoDB, Redis compose
compose.prod.yaml	운영 Frontend/Nginx, Backend, Redis, Certbot compose
compose.ai.yaml	AI API, AI worker, AI Redis compose
Jenkinsfile	CI/CD 파이프라인
FE/Dockerfile	프론트 빌드 및 Nginx runtime 이미지

파일	설명
FE/nginx.conf	운영 Nginx HTTPS/proxy 설정
BE/Dockerfile	백엔드 Gradle build 및 Java runtime 이미지
BE/src/main/resources/application.yaml	백엔드 설정
BE/src/main/resources/db/migration	Flyway migration
AI/Dockerfile	AI API/worker 공통 이미지
AI/app/core/config.py	AI 환경 변수 설정

DB 접속 정보가 정의된 파일 목록

파일	내용
.env.prod	운영 MySQL/MongoDB/Redis 접속 정보, JWT/OAuth/S3 키 (서버에만 보관)
.env	개발 MySQL/MongoDB/Redis 접속 정보 (로컬 전용)
.env.ai	AI 서버 MySQL/MongoDB 접속 정보, LLM API 키

15. 빌드 환경 변수 및 배포 특이사항

15.1 빌드 시 사용되는 환경 변수

프론트엔드 (빌드 타임 주입)

프론트엔드의 VITE_ 환경 변수는 FE/Dockerfile 의 ARG 로 선언되어 빌드 시 정적으로 번들에 삽입된다. 배포 후 URL을 변경하려면 반드시 이미지를 재빌드해야 한다.

변수	전달 경로	설명
VITE_API_BASE_URL	.env.prod → compose.prod.yaml build arg → Docker ARG → 번들 삽입	REST API 기본 URL (예: https://studion.ai.kr)
VITE_WS_BASE_URL	동일	WebSocket 기본 URL (예: wss://studion.ai.kr)

로컬 개발 시 FE/.env.local 을 생성해 값을 오버라이드할 수 있다. 미설정 시 각각 http://localhost:8080 , ws://localhost:8080 이 기본값으로 사용된다.

백엔드 (런타임 주입)

백엔드는 빌드 타임 환경 변수를 사용하지 않는다. Gradle 빌드는 환경과 독립적이며, 모든 설정 값은 컨테이너 실행 시 .env.prod 에서 런타임에 주입된다. 빌드 명령은 섹션 11.1 참고.

AI 서버 (런타임 주입)

AI 서버도 빌드 타임 환경 변수를 사용하지 않는다. `uv sync --frozen` 으로 고정된 lockfile 기준으로 의존성을 설치하며, 설정 값은 컨테이너 실행 시 `.env.ai` 에서 런타임에 주입된다.

15.2 배포 시 특이사항

1. 프론트엔드 URL 변경 시 재빌드 필수

`VITE_API_BASE_URL`, `VITE_WS_BASE_URL` 은 빌드 시 번들에 삽입된다. 도메인 변경 시 `compose.prod.yaml` build 를 다시 실행해야 한다.

2. Redis password 일치 확인

`compose.prod.yaml` 의 redis 서비스는 `sh -c 'redis-server --requirepass "$$REDIS_PASSWORD"'` 명령으로 실행된다. `.env.prod` 의 `REDIS_PASSWORD` 값이 백엔드의 `REDIS_PASSWORD` 와 동일해야 한다.

3. 백엔드는 Redis healthcheck 후 시작

`compose.prod.yaml` 에서 backend 서비스는 `depends_on: redis: condition: service_healthy` 로 설정되어 있다. Redis가 정상 응답하기 전까지 백엔드 컨테이너가 시작되지 않는다.

4. spring.profiles.include: secret 처리

`application.yaml` 에 `spring.profiles.include: secret` 설정이 있으나 `application-secret.yaml` 파일은 존재하지 않는다. 모든 민감 정보는 환경 변수로 주입되므로 별도 파일 생성은 불필요하다.

5. Nginx reverse proxy + HTTPS 헤더 처리

`compose.prod.yaml` 의 backend 서비스에 `SERVER_FORWARD_HEADERS_STRATEGY=framework` 환경 변수가 고정 설정되어 있다. Nginx reverse proxy 뒤에서 Spring Security의 OAuth2 redirect와 HTTPS 처리가 올바르게 동작하기 위해 필요하다.

6. FFmpeg 백엔드 이미지 내장

BE/Dockerfile 에서 `apt-get install ffmpeg` 로 FFmpeg를 이미지에 포함한다. 백엔드 오디오 렌더링에 사용되며 서버에 별도 설치 불필요하다.

7. AI worker 3개 인스턴스 실행

운영 배포 시 `--scale ai-worker=3` 으로 워커를 3개 실행한다. Dramatiq가 Redis 브로커를 통해 작업을 분배한다. 개발 환경에서는 1개만 실행해도 무방하다.

16. 외부 서비스 정보

프로젝트에서 사용하는 외부 서비스 목록과 가입/설정에 필요한 정보다.

16.1 Google OAuth 2.0 (필수)

- 목적: 소셜 로그인 (유일한 인증 수단)
- 가입: [Google Cloud Console](#) → API 및 서비스 → 사용자 인증 정보
- 설정 절차:
 1. OAuth 2.0 클라이언트 ID 생성 (웹 애플리케이션)
 2. 승인된 리디렉션 URI 등록: `https://<도메인>/login/oauth2/code/google`
 3. 개발 환경 추가 URI: `http://localhost:8080/login/oauth2/code/google`

- **필요한 값:** `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`
- **참고:** `BE/src/main/resources/application.yaml` 의 `spring.security.oauth2` 섹션

16.2 AWS S3 (필수)

- **목적:** 오디오 파일 업로드 및 저장
- **가입:** [AWS Console](#) → S3
- **설정 절차:**
 1. S3 버킷 생성 (리전: `ap-northeast-2` 서울)
 2. IAM 사용자 생성, 정책 부여: `s3:GetObject`, `s3:PutObject`, `s3:DeleteObject`
 3. 버킷 CORS 설정: 프론트엔드 도메인에서의 PUT 요청 허용
 4. CDN 도메인 설정 (CloudFront 또는 S3 도메인 직접 사용)
- **필요한 값:** `AWS_ACCESS_KEY`, `AWS_SECRET_KEY`, `AWS_S3_BUCKET`, `CDN_DOMAIN`
- **참고:** `BE/src/main/resources/application.yaml` 의 `cloud.aws` 섹션

16.3 Let's Encrypt / Certbot (필수)

- **목적:** HTTPS SSL 인증서 자동 발급
- **별도 가입 불필요:** 도메인 DNS A 레코드가 서버를 가리키도록 설정 후 Certbot 실행
- **필요한 값:** `CERTBOT_EMAIL` (만료 알림 수신용), `CERTBOT_DOMAIN`
- **발급 명령:** 섹션 7.1 참고
- **갱신:** Certbot은 90일 유효 인증서를 발급한다. 정기적인 갱신 cron 설정을 권장한다.

16.4 CLAP 추론 서버 (선택)

- **목적:** 오디오 보컬/악기 분류 AI 분석 (AI 워크플로우 핵심 기능)
- **형태:** 별도 ML 서버 (자체 구축 또는 외부 API)
- **비활성화:** `STUDION_AI_CLAP_ENABLED=false` 설정 시 해당 분석 단계를 건너뛴
- **필요한 값:** `STUDION_AI_CLAP_INFERENCE_URL` (활성화 시)
- **API 형식:** POST 요청으로 오디오 데이터를 전달하면 분류 결과를 반환하는 HTTP 서버

16.5 Planning LLM API (선택)

- **목적:** AI 워크플로우의 작업 계획 수립
- **기본 상태:** `STUDION_AI_PLANNING_LLM_ENABLED=false` (비활성화)
- **호환:** OpenAI API 호환 엔드포인트 (OpenAI, Azure OpenAI, 로컬 모델 등)
- **필요한 값:** `STUDION_AI_PLANNING_LLM_ENABLED=true`,
`STUDION_AI_PLANNING_LLM_BASE_URL`, `STUDION_AI_PLANNING_LLM_API_KEY`,
`STUDION_AI_PLANNING_LLM_MODEL`

16.6 Plan Critic LLM API (선택)

- **목적:** AI 워크플로우 계획의 품질 검증
- **기본 상태:** `STUDION_AI_PLAN_CRITIC_ENABLED=false` (비활성화)

- **호환:** Anthropic API
- **필요한 값:** STUDIO_AI_PLAN_CRITIC_ENABLED=true ,
STUDIO_AI_PLAN_CRITIC_BASE_URL , STUDIO_AI_PLAN_CRITIC_API_KEY ,
STUDIO_AI_PLAN_CRITIC_MODEL
- **유효한 모델 예시:** claude-sonnet-4-6 , claude-haiku-4-5-20251001

16.7 Google Analytics (선택)

- **목적:** 사용자 행동 분석 (페이지뷰, 이벤트 추적)
- **참고:** FE/src/shared/lib/gtag.ts 및 FE/src/app/router.ts
- **설정:** Google Analytics 4 속성 생성 후 측정 ID를 프론트엔드 코드에 반영

17. DB 덤프

17.1 MySQL 전체 스키마 DDL

JPA 엔티티(BE/src/main/java)를 기반으로 생성한 CREATE TABLE 문이다. 신규 환경 구성 시 아래 DDL을 먼저 적용한 뒤 백엔드를 기동하면 Flyway가 V1~V3 마이그레이션을 자동 적용한다.

```
-- =====
-- STUDIO MySQL 전체 스키마 DDL
-- charset: utf8mb4
-- =====

SET NAMES utf8mb4;

-- 1. position_group
CREATE TABLE IF NOT EXISTS position_group (
    code          INT          NOT NULL,
    name          VARCHAR(50)  NOT NULL,
    `order`       INT          NOT NULL,
    created_at    TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    updated_at    TIMESTAMP    NULL,
    created_by    INT          NULL,
    updated_by    INT          NULL,
    PRIMARY KEY (code)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 2. position_detail
CREATE TABLE IF NOT EXISTS position_detail (
    code          INT          NOT NULL,
    group_code    INT          NULL,
    name          VARCHAR(50)  NOT NULL,
    `order`       INT          NOT NULL,
    created_at    TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    updated_at    TIMESTAMP    NULL,
    created_by    INT          NULL,
    updated_by    INT          NULL,
```

```

        PRIMARY KEY (code),
        CONSTRAINT fk_pd_group FOREIGN KEY (group_code) REFERENCES
position_group (code)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 3. user
CREATE TABLE IF NOT EXISTS user (
    id                INT                NOT NULL AUTO_INCREMENT,
    email             VARCHAR(255) NOT NULL,
    provider          VARCHAR(20)  NOT NULL,
    provider_id       VARCHAR(100) NOT NULL,
    profile_img_url   VARCHAR(255) NULL,
    nickname          VARCHAR(20)  NOT NULL,
    role              VARCHAR(20)  NOT NULL DEFAULT 'BASIC', -- BASIC | ADMIN
    last_login_at     TIMESTAMP     NOT NULL,
    deleted_at        TIMESTAMP     NULL,
    created_at        TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    updated_at        TIMESTAMP     NULL,
    created_by        INT           NULL,
    updated_by        INT           NULL,
    PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 4. user_position (복합 PK)
CREATE TABLE IF NOT EXISTS user_position (
    user_id          INT NOT NULL,
    position_code    INT NOT NULL,
    created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at       TIMESTAMP NULL,
    created_by       INT NULL,
    updated_by       INT NULL,
    PRIMARY KEY (user_id, position_code),
    CONSTRAINT fk_up_user FOREIGN KEY (user_id) REFERENCES user
(id),
    CONSTRAINT fk_up_detail FOREIGN KEY (position_code) REFERENCES
position_detail (code)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 5. project
CREATE TABLE IF NOT EXISTS project (
    id                INT                NOT NULL AUTO_INCREMENT,
    name              VARCHAR(100) NOT NULL,
    root_note         VARCHAR(10)  NOT NULL DEFAULT 'C', --
C|C_SHARP|D|E_FLAT|E|F|F_SHARP|G|A_FLAT|A|B_FLAT|B
    mode              VARCHAR(10)  NOT NULL DEFAULT 'MAJOR', -- MAJOR |
MINOR
    tempo            DOUBLE         NOT NULL DEFAULT 120.0,
    time_sig_numerator INT          NOT NULL DEFAULT 4,
    time_sig_denominator INT        NOT NULL DEFAULT 4,
    total_bar_count  INT           NOT NULL DEFAULT 0,

```

```

total_play_time_ms    INT          NOT NULL DEFAULT 0,
track_count           INT          NOT NULL DEFAULT 0,
total_audio_size_byte BIGINT       NOT NULL DEFAULT 0,
deleted_at            TIMESTAMP    NULL,
last_update_at        TIMESTAMP    NULL,
created_at            TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
updated_at            TIMESTAMP    NULL,
created_by            INT          NULL,
updated_by            INT          NULL,
PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 6. project_member
CREATE TABLE IF NOT EXISTS project_member (
  id          INT NOT NULL AUTO_INCREMENT,
  project_id  INT NOT NULL,
  user_id     INT NOT NULL,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP NULL,
  created_by  INT NULL,
  updated_by  INT NULL,
  PRIMARY KEY (id),
  CONSTRAINT uk_pm_project_user UNIQUE (project_id, user_id),
  CONSTRAINT fk_pm_project FOREIGN KEY (project_id) REFERENCES project
(id),
  CONSTRAINT fk_pm_user FOREIGN KEY (user_id) REFERENCES user (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 7. audio_metadata
CREATE TABLE IF NOT EXISTS audio_metadata (
  id          INT          NOT NULL AUTO_INCREMENT,
  object_key   VARCHAR(1024) NOT NULL,
  original_name VARCHAR(255) NOT NULL,
  stored_name  VARCHAR(255) NOT NULL,
  mime_type    VARCHAR(255) NULL, -- MPEG | WAV
  size_bytes   INT          NOT NULL,
  duration_ms  INT          NOT NULL,
  deleted_at   TIMESTAMP    NULL,
  created_at   TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
  updated_at   TIMESTAMP    NULL,
  created_by   INT          NULL,
  updated_by   INT          NULL,
  PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 8. project_master_audio_version
CREATE TABLE IF NOT EXISTS project_master_audio_version (
  id          INT          NOT NULL AUTO_INCREMENT,
  project_id  INT          NOT NULL,
  audio_metadata_id INT          NULL, -- V2에서 nullable로 변경

```

```

name          VARCHAR(50)   NOT NULL,
memo          VARCHAR(255)  NULL,
status        VARCHAR(255)  NULL, -- PROCESSING | READY | FAILED
failed_reason VARCHAR(1000)  NULL,
render_snapshot_json LONGTEXT  NULL,
created_at    TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
updated_at    TIMESTAMP     NULL,
created_by    INT           NULL,
updated_by    INT           NULL,
PRIMARY KEY (id),
CONSTRAINT fk_pmav_project FOREIGN KEY (project_id) REFERENCES
project (id),
CONSTRAINT fk_pmav_audio FOREIGN KEY (audio_metadata_id) REFERENCES
audio_metadata (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

-- 9. master_track (project와 1:1)

```

CREATE TABLE IF NOT EXISTS master_track (
    project_id INT          NOT NULL,
    is_soloed  TINYINT(1)   NOT NULL DEFAULT 0,
    is_muted   TINYINT(1)   NOT NULL DEFAULT 0,
    volume     DOUBLE       NOT NULL DEFAULT 0.0,
    pan        INT          NOT NULL DEFAULT 0,
    created_at TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP     NULL,
    created_by INT          NULL,
    updated_by INT          NULL,
    PRIMARY KEY (project_id),
    CONSTRAINT fk_mt_project FOREIGN KEY (project_id) REFERENCES project
(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

-- 10. track (id는 애플리케이션에서 직접 관리, AUTO_INCREMENT 아님)

```

CREATE TABLE IF NOT EXISTS track (
    id          INT          NOT NULL,
    project_id  INT          NOT NULL,
    pre_track_id INT        NULL,
    post_track_id INT       NULL,
    track_type  VARCHAR(255) NOT NULL DEFAULT 'AUDIO', -- AUDIO | MIDI
    name        VARCHAR(255) NULL,
    is_soloed   TINYINT(1)   NOT NULL DEFAULT 0,
    is_muted    TINYINT(1)   NOT NULL DEFAULT 0,
    volume      DOUBLE       NOT NULL DEFAULT 0.0,
    pan         INT          NOT NULL DEFAULT 0,
    created_at  TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    updated_at  TIMESTAMP     NULL,
    created_by  INT          NULL,
    updated_by  INT          NULL,
    PRIMARY KEY (id),
    CONSTRAINT fk_track_project FOREIGN KEY (project_id) REFERENCES project

```

```
(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- 11. clip (id는 애플리케이션에서 직접 관리)

```
CREATE TABLE IF NOT EXISTS clip (
  id INT NOT NULL,
  track_id INT NOT NULL,
  audio_metadata_id INT NOT NULL,
  color VARCHAR(7) NOT NULL DEFAULT '#FFFFFF',
  start DOUBLE NOT NULL DEFAULT 1.0,
  duration DOUBLE NOT NULL DEFAULT 1.0,
  audio_start_ms INT NOT NULL DEFAULT 0,
  audio_duration_ms INT NOT NULL DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NULL,
  created_by INT NULL,
  updated_by INT NULL,
  PRIMARY KEY (id),
  CONSTRAINT fk_clip_track FOREIGN KEY (track_id) REFERENCES
track (id),
  CONSTRAINT fk_clip_audio FOREIGN KEY (audio_metadata_id) REFERENCES
audio_metadata (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- 12. comment (id는 애플리케이션에서 직접 관리)

```
CREATE TABLE IF NOT EXISTS comment (
  id INT NOT NULL,
  track_id INT NULL,
  user_id INT NULL,
  parent_comment_id INT NULL,
  content VARCHAR(255) NOT NULL,
  location DECIMAL(11, 7) NOT NULL,
  is_resolved TINYINT(1) NOT NULL DEFAULT 0,
  deleted_at TIMESTAMP NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NULL,
  created_by INT NULL,
  updated_by INT NULL,
  PRIMARY KEY (id),
  CONSTRAINT fk_comment_track FOREIGN KEY (track_id) REFERENCES track
(id),
  CONSTRAINT fk_comment_user FOREIGN KEY (user_id) REFERENCES user (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- 13. comment_mention (복합 PK)

```
CREATE TABLE IF NOT EXISTS comment_mention (
  comment_id INT NOT NULL,
  user_id INT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NULL,
```

```

        created_by INT NULL,
        updated_by INT NULL,
        PRIMARY KEY (comment_id, user_id),
        CONSTRAINT fk_cm_comment FOREIGN KEY (comment_id) REFERENCES comment
(id),
        CONSTRAINT fk_cm_user FOREIGN KEY (user_id) REFERENCES user (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

-- 14. track_eq

```

CREATE TABLE IF NOT EXISTS track_eq (
    id INT NOT NULL AUTO_INCREMENT,
    track_id INT NOT NULL,
    project_id INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL,
    created_by INT NULL,
    updated_by INT NULL,
    PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

-- 15. track_eq_band

```

CREATE TABLE IF NOT EXISTS track_eq_band (
    id INT NOT NULL AUTO_INCREMENT,
    track_eq_id INT NOT NULL,
    band_order INT NOT NULL,
    eq_type_code INT NOT NULL,
    frequency_hz INT NOT NULL,
    q DOUBLE NOT NULL,
    gain_delta_db DOUBLE NOT NULL,
    job_id INT NULL,
    suggestion_action_id INT NULL,
    applied_suggestion_id INT NULL,
    source_type_code INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL,
    created_by INT NULL,
    updated_by INT NULL,
    PRIMARY KEY (id),
    CONSTRAINT fk_teb_eq FOREIGN KEY (track_eq_id) REFERENCES track_eq (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

-- 16. master_limiter

```

CREATE TABLE IF NOT EXISTS master_limiter (
    id INT NOT NULL AUTO_INCREMENT,
    project_id INT NOT NULL UNIQUE,
    is_enabled TINYINT(1) NOT NULL DEFAULT 0,
    threshold_db DOUBLE NOT NULL DEFAULT -6.0,
    ceiling_dbfs DOUBLE NOT NULL DEFAULT -1.0,
    attack_ms DOUBLE NOT NULL DEFAULT 3.0,
    release_ms DOUBLE NOT NULL DEFAULT 80.0,

```



```

input_gain_db          DOUBLE      NOT NULL DEFAULT 0.0,
makeup_gain_db         DOUBLE      NOT NULL DEFAULT 0.0,
job_id                 INT          NULL,
suggestion_action_id   INT          NULL,
applied_suggestion_id  INT          NULL,
source_type_code       INT          NOT NULL DEFAULT 1,
created_at             TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
updated_at             TIMESTAMP    NULL,
created_by             INT          NULL,
updated_by             INT          NULL,
PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 17. ai_analysis_job
CREATE TABLE IF NOT EXISTS ai_analysis_job (
  id                  INT          NOT NULL AUTO_INCREMENT,
  project_id          INT          NOT NULL,
  requested_by        INT          NULL,
  dispatch_type       VARCHAR(50)  NULL,
  status              VARCHAR(50)  NOT NULL DEFAULT 'QUEUED',
  phase               VARCHAR(100) NOT NULL DEFAULT 'DISPATCHING',
  current_node        VARCHAR(100) NULL,
  progress            INT          NOT NULL DEFAULT 0,
  queue_name          VARCHAR(100) NULL,
  timeline_snapshot_id VARCHAR(255) NULL,
  started_at          TIMESTAMP    NULL,
  completed_at        TIMESTAMP    NULL,
  error_code          VARCHAR(100) NULL,
  error_message        VARCHAR(1000) NULL,
  created_at          TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
  updated_at          TIMESTAMP    NULL,
  created_by          INT          NULL,
  updated_by          INT          NULL,
  PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 18. dm_room
CREATE TABLE IF NOT EXISTS dm_room (
  id          INT NOT NULL AUTO_INCREMENT,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP NULL,
  created_by  INT NULL,
  updated_by  INT NULL,
  PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 19. dm_room_member
CREATE TABLE IF NOT EXISTS dm_room_member (
  id          INT NOT NULL AUTO_INCREMENT,
  dm_room_id  INT NULL,

```

```

user_id      INT NULL,
created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at   TIMESTAMP NULL,
created_by   INT NULL,
updated_by   INT NULL,
PRIMARY KEY (id),
CONSTRAINT uk_drm_user_room UNIQUE (user_id, dm_room_id),
CONSTRAINT fk_drm_room FOREIGN KEY (dm_room_id) REFERENCES dm_room (id),
CONSTRAINT fk_drm_user FOREIGN KEY (user_id) REFERENCES user (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

17.2 Flyway 마이그레이션 (V1~V3 시드 데이터)

DDL 적용 후 백엔드를 기동하면 Flyway가 아래 마이그레이션을 자동 실행한다. 수동 적용이 필요한 경우 아래 SQL을 순서대로 실행한다.

파일	내용
V1__seed_position_data.sql	position_group, position_detail 시드 데이터 삽입
V2__alter_project_master_audio_version_for_audio_version_render.sql	audio_metadata nullable 변경
V3__add_role_to_user.sql	user.role 추가 (기본값 0)

```

-- V1: position_group / position_detail 시드 데이터
INSERT IGNORE INTO position_group (code, name, `order`) VALUES
  (1, 'composer', 1), (2, 'producer', 2), (3, 'dj', 3), (4, 'sound engineer', 4),
  (5, 'vocal', 5), (6, 'keys', 6), (7, 'guitar', 7), (8, 'bass', 8), (9, 'drum', 9),
  (10, 'perc', 10), (11, 'string', 11), (12, 'woodwind', 12), (13, 'brass', 13),
  (14, 'etc', 14);

INSERT IGNORE INTO position_detail (code, group_code, name, `order`) VALUES
  (1, 1, 'composer', 1), (2, 2, 'producer', 1), (3, 3, 'dj', 1), (4, 4, 'sound
engineer', 1),
  (5, 5, 'vocal', 1), (6, 6, 'piano', 1), (7, 6, 'keyboard', 2), (8, 6, 'organ', 3),
  (9, 6, 'etc', 4),
  (10, 7, 'classical guitar', 1), (11, 7, 'electric guitar', 2), (12, 7, 'acoustic
guitar', 3), (13, 7, 'etc', 4),
  (14, 8, 'upright bass', 1), (15, 8, 'electric bass', 2), (16, 8, 'etc', 3),
  (17, 9, 'drum', 1), (18, 10, 'percussion', 1),
  (19, 11, 'violin', 1), (20, 11, 'viola', 2), (21, 11, 'cello', 3), (22, 11, 'double
bass', 4), (23, 11, 'harp', 5), (24, 11, 'etc', 6),
  (25, 12, 'piccolo', 1), (26, 12, 'flute', 2), (27, 12, 'oboe', 3),
  (28, 12, 'clarinet', 4), (29, 12, 'bassoon', 5), (30, 12, 'saxophone', 6),
  (31, 12, 'etc', 7),

```

```

(32,13,'french horn',1),(33,13,'trumpet',2),(34,13,'trombone',3),
(35,13,'tuba',4),(36,13,'etc',5),
(37,14,'etc',1);

-- V2: audio_metadata_id nullable 변경
ALTER TABLE project_master_audio_version
    MODIFY COLUMN audio_metadata_id INT NULL;

-- V3: user.role 컬럼 추가
ALTER TABLE user ADD COLUMN role VARCHAR(20) NULL;
UPDATE user SET role = 'BASIC' WHERE role IS NULL;
ALTER TABLE user MODIFY COLUMN role VARCHAR(20) NOT NULL DEFAULT 'BASIC';

```

17.3 운영 DB 전체 덤프 생성

운영 데이터가 포함된 전체 덤프가 필요한 경우 아래 명령을 실행한다. 덤프 파일은 Git에 포함하지 않고 별도 보안 경로에 보관한다.

```

mysqldump -h <MYSQL_HOST> -u <DB_USERNAME> -p<DB_PASSWORD> \
    --single-transaction --routines --triggers \
    <MYSQL_DATABASE> > studio_$(date +%Y%m%d).sql

```

17.4 MongoDB

MongoDB는 애플리케이션 실행 시 컬렉션이 자동 생성되므로 별도 스키마 적용이 불필요하다. 운영 데이터 덤프가 필요한 경우:

```

mongodump \
    --uri="mongodb://<MONGO_USERNAME>:
<MONGO_PASSWORD>@<MONGO_HOST>:27017/<MONGODB_DB>?authSource=admin" \
    --out=dump_$(date +%Y%m%d)

# 복원
mongorestore \
    --uri="mongodb://<MONGO_USERNAME>:
<MONGO_PASSWORD>@<MONGO_HOST>:27017/<MONGODB_DB>?authSource=admin" \
    dump_YYYYMMDD/<MONGODB_DB>/

```

18. 시연 시나리오

사전 준비

- Google 계정 2개: 대표 시연자(피아노/편집 역할), 보조 시연자(드럼 역할)
- 보조 시연자는 시연 전 미리 프로젝트에 입장해 대기
- 시연 프로젝트에 베이스·키보드 트랙 오디오 파일 업로드 완료 상태로 준비
- 시연용 오디오 파일: Over the Horizon 멜로디 트랙(대표), 드럼 트랙(보조)

18.0 오프닝

화면 동작 없음. 대표 시연자 대사.

실용음악과 1학년 페르소나로 팀플 과제 편곡 작업을 소개한다. 익숙한 멜로디를 자기 색깔로 편곡해 SNS에 올리는 것이 목표임을 설명한다.

18.1 프로젝트 입장

순서	시연자	화면	동작
1	보조	/project/:projectId	미리 프로젝트에 입장해 대기 — 타임라인에 베이스·키보드 트랙이 보이는 상태
2	대표	/dashboard	대시보드에서 해당 프로젝트 카드 클릭
3	대표	/project/:projectId	프로젝트 편집 화면 진입 — 헤더 온라인 유저 목록에 보조 시연자 아바타가 표시됨을 확인

"예전엔 파일을 주고받는 무한 반복이었지만, 오늘은 studiON에서 바로 같이 작업합니다."

18.2 메인 멜로디 업로드

순서	시연자	화면	동작
1	대표	트랙 영역	빈 트랙의 파일 선택 버튼 클릭 → Over the Horizon 멜로디 오디오 파일 선택
2	대표	타임라인	업로드 완료 후 멜로디 클립의 파형(waveform)이 타임라인에 표시됨 확인
3	대표	재생 컨트롤러	재생(▶) 버튼 클릭 → 멜로디 + 베이스 + 키보드 합주 미리듣기

"업로드하면 바로 타임라인에 올라오고, 팀원들도 같은 화면에서 바로 확인할 수 있습니다."

18.3 보조 사운드 업로드

순서	시연자	화면	동작
1	보조	트랙 영역	빈 트랙의 파일 선택 버튼 클릭 → 드럼 트랙 오디오 파일 선택
2	대표	타임라인	보조 시연자가 업로드한 드럼 클립이 실시간으로 대표 화면에도 표시됨 확인
3	대표	재생 컨트롤러	재생(▶) 버튼 클릭 → 전체 4트랙 합주 미리듣기

"드럼 친구도 올려줬네요! 드럼까지 합친 버전으로 들어보겠습니다."

18.4 소리 충돌 문제 확인

순서	시연자	화면	동작
1	대표	재생 컨트롤러 / 마스터 트랙	전체 재생 중 마스터 출력 레벨 미터 또는 클리핑 표시를 시각적으로 보여줌

"여러 사운드가 쌓이면 각 트랙은 괜찮아도 같이 들었을 때 소리가 답답하거나 깨지는 구간이 생길 수 있어요. 어느 트랙의 어느 구간인지 체크하는 데 시간이 꽤 많이 걸리는데, 그 시간이 제일 아깝더라고요."

18.5 AI 분석 실행

순서	시연자	화면	동작
1	대표	우측 사이드 패널	"AI" 탭 클릭
2	대표	AI 패널	AI 분석 시작 버튼 클릭
3	대표	AI 패널	"분석 중" 상태 표시 확인
4	대표	/dashboard	분석 대기(약 90초) 동안 대시보드로 이동해 다른 프로젝트로 전환

"AI가 제 음악을 대신 완성해주는 게 아니라, 먼저 확인해야 할 문제 후보 구간을 찾아주는 역할이에요. 분석하는 동안 다른 프로젝트로 이어서 진행하겠습니다."

18.6 팀원 댓글 피드백

순서	시연자	화면	동작
1	보조	타임라인	멜로디 트랙의 특정 구간 클릭 → 댓글 입력창 열림
2	보조	댓글 입력창	"지금도 좋은데 다른 느낌으로 리믹스도 해보는 건 어떨까요?" 입력 후 등록
3	대표	우측 사이드 패널	"댓글" 탭 클릭 → 보조 시연자가 작성한 댓글 확인
4	대표	댓글 목록	댓글 항목 클릭 → 타임라인이 해당 구간으로 이동함 확인

"기존에는 다른 메시지 앱을 이용해야 했고 텍스트로만 표현할 수 있었는데, 여기서는 댓글을 특정 구간에 바로 붙일 수 있어서 편리해요."

18.7 피드백 반영 — 리믹스 편집

순서	시연자	화면	동작
1	대표	타임라인	멜로디 클립 우클릭(또는 메뉴) → "복사" 선택
2	대표	타임라인	원하는 위치에 "붙여넣기" → 반복 1회 더 실행해 클립 3개로 구성
3	대표	재생 컨트롤러	재생(▶) 버튼 클릭 → 리믹스 버전 미리듣기
4	보조	댓글 목록	대표 시연자의 댓글에 대댓글로 좋아요/긍정 반응 작성
5	대표	댓글 목록	보조 시연자의 대댓글 확인

"피드백을 반영해서 클립을 복붙해 리믹스처럼 만들어봤어요. 댓글 단 친구도 만족한 것 같네요."

18.8 AI 분석 결과 확인 및 적용

순서	시연자	화면	동작
1	대표	/dashboard	앞서 분석을 실행한 프로젝트 카드 클릭 → 프로젝트 재진입
2	대표	우측 사이드 패널	"AI" 탭 클릭 → 분석 완료 결과 확인

순서	시연 자	화면	동작
3	대표	AI 분석 결과 오버레이	감지된 구간 목록 확인 — 각 구간별 이상 항목(클리핑 등)과 제안 내용 표시
4	대표	AI 분석 결과 오버레이	7
5	대표	AI 패널 또는 EQ 패널	프롬프트 입력창에 보정 목표 입력 → 자동 EQ 보정 실행
6	대표	재생 컨트롤러	Before(적용 전) 재생 후 After(적용 후) 재생 → 차이 비교

"굉장히 많은 구간에서 이상이 감지됐어요. 일부분만 AI 제안대로 적용해볼게요. 프롬프팅으로 자동 EQ 보정도 가능합니다."

18.9 완성본 내보내기

순서	시연 자	화면	동작
1	대표	프로젝트 편집 화면 헤더	"내보내기" 버튼 클릭 → 내보내기 모달 열림
2	대표	내보내기 모달	출력 형식 선택 후 "내보내기" 버튼 클릭
3	대표	내보내기 모달	렌더링 완료 후 다운로드 버튼 클릭 → 오디오 파일 저장
4	대표	로컬 플레이어 또는 재생 컨트롤러	완성된 음원 파일 재생

"마지막으로 결과물을 음원 파일로 내보낼게요. 스튜디오 덕분에 이번 팀플도 기대해볼 수 있겠어요! 여기까지 시연을 맡은 한예성이었습니다."